Comprix: Relazione del progetto

Relazione e progetto di Giuseppe Mangano (mat. XXXXXX).

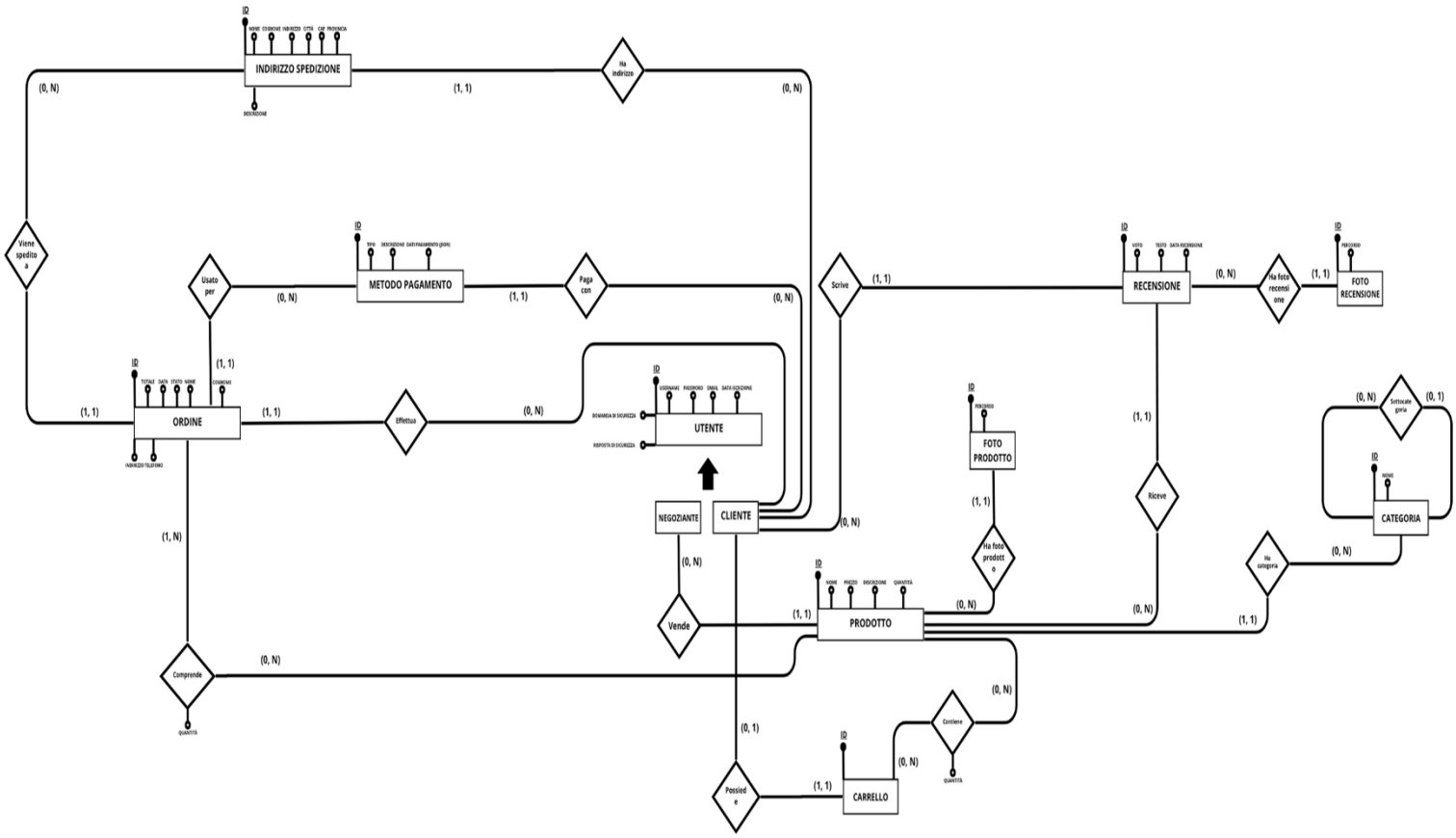
Progetto per l'esame di Basi di Dati e Laboratorio di Programmazione Web, a.a. 2024/2025.

Introduzione

Comprix è un'applicazione web per la gestione di un servizio di e-commerce. Questa piattaforma consente ai negozianti di registrarsi e gestire un proprio catalogo di prodotti, e ai clienti di cercare, acquistare e recensire tali prodotti. L'intero sistema è stato sviluppato seguendo le specifiche fornite; non è stato applicato il pattern Model View Controller perché è un pattern che si addice meglio alla programmazione ad oggetti; data la natura procedurale del progetto, è stato seguito un pattern simile, ovvero, la parte front-end è la "view", mentre la parte che gestisce le operazioni back-end fa sia da "model" che da "controller". L'interazione con la base di dati è gestita tramite un database relazionale con MariaDB, mentre l'applicazione web è costruita in PHP, HTML, CSS e JavaScript (con uso di jQuery e di AJAX). Questa relazione illustra le scelte progettuali e implementative del progetto, con un'analisi della struttura del database, delle funzionalità implementate e delle misure di sicurezza adottate. L'homepage del sito è visualizzabile da chiunque, sia dagli utenti non registrati sia dagli utenti registrati.

Parte I: La base di dati

I. Il diagramma E-R



II. Analisi delle ridondanze

Nello schema logico sono state introdotte quattro ridondanze intenzionali, analizzate di seguito per valutarne la convenienza.

La formula di riferimento utilizzata è:

$C(O) = F(O) \times w_t (a \times NA_w + NA_r)$

I parametri assunti sono:

- $F(O) = 1000$ (frequenza giornaliera visualizzazioni)
- $w_t = 1$ (operazione interattiva)
- $a = 3$ (coefficiente relativo al costo di una scrittura)
- Prodotti medi per carrello/ordine: 5

Prima ridondanza: campi calcolati nel carrello (totale_elementi, totale_prezzo)

Questa ridondanza evita di dover leggere e sommare tutti i record ogni volta che si visualizza l'icona del carrello o la pagina di riepilogo.

Operazione: visualizzazione del totale del carrello.

Schema senza ridondanza (S):

Per ottenere il totale, il sistema deve leggere la tabella carrello (1 accesso) e poi scansionare la tabella elemento_carrello per sommare prezzi e quantità dei 5 prodotti (5 accessi), più l'accesso alla tabella prodotto per il prezzo unitario attuale (5 accessi).

- $NA_r = 1 + 5 + 5 = 11$
- $NA_w = 0$
- $C(S) = 1000 \times 1 \times (3 \times 0 + 11) = 11.000$

Schema con ridondanza (S_{rid}):

I totali sono già presenti nella tabella carrello. È sufficiente leggere quel singolo record.

- $NA_r = 1$
- $NA_w = 0$
- $C(S_{rid}) = 1000 \times 1 \times (3 \times 0 + 1) = 1.000$

Risultato: La ridondanza offre un abbattimento dei costi di lettura di 11 volte ($\Delta C = 11$).

Seconda ridondanza: snapshot storico in elemento_ordine (nome_prodotto, prezzo_unitario)

Questa ridondanza "congela" i dati del prodotto al momento dell'acquisto per integrità storica, ma ha anche un impatto sulle performance della visualizzazione dello storico degli ordini.

Operazione: visualizzazione dei dettagli di un ordine passato.

Schema senza ridondanza (S):

Si legge l'ordine (1 accesso), poi si recuperano le 5 righe di elemento_ordine (5 accessi) e per ognuna si deve fare join con la tabella prodotto per recuperare nome e prezzo (5 accessi).

- $NA_r = 1 + 5 + 5 = 11$
- $NA_w = 0$
- $C(S) = 1000 \times 1 \times (3 \times 0 + 11) = 11.000$

Schema con ridondanza (S_{rid}):

Si legge l'ordine (1 accesso) e le righe di elemento_ordine (5 accessi). I dati (nome e prezzo) sono già lì, non serve accedere alla tabella prodotto.

- $NA_r = 1 + 5 = 6$
- $NA_w = 0$
- $C(S_{rid}) = 1000 \times 1 \times (3 \times 0 + 6) = 6.000$

Risultato: Oltre a garantire la correttezza dei dati storici, la ridondanza quasi dimezza il costo operazionale di lettura ($\Delta C = 1,8$).

Terza ridondanza: snapshot dell'indirizzo in ordine (indirizzo_destinatario, citta, ecc.)

L'indirizzo viene copiato nella tabella ordine anziché mantenere solo il riferimento all'ID dell'indirizzo del profilo utente.

Operazione: Visualizzazione della spedizione di un ordine.

Schema senza ridondanza (S):

Si legge la tabella ordine (1 accesso) e si deve fare JOIN con la tabella indirizzo_spedizione (1 accesso) per mostrare dove spedire.

- $NA_r = 1 + 1 = 2$
- $NA_w = 0$
- $C(S) = 1000 \times 1 \times (3 \times 0 + 2) = 2.000$

Schema con ridondanza (S_{rid}):

Tutti i dati necessari sono nella riga della tabella ordine.

- $NA_r = 1$
- $NA_w = 0$
- $C(S_{rid}) = 1000 \times 1 \times (3 \times 0 + 1) = 1.000$

Risultato: La ridondanza dimezza il costo operazionale di lettura ($\Delta C = 2$), eliminando la necessità di join con la tabella indirizzi (che potrebbe anche non contenere più quel record se l'utente lo cancellasse).

Quarta ridondanza: denormalizzazione id_negoziante in elemento_ordine

L'id_negoziante è copiato in elemento_ordine anche se sarebbe derivabile tramite la relazione con prodotto.

Operazione: Recupero del venditore per una riga d'ordine

Schema senza ridondanza (S):

Si legge elemento_ordine (1 accesso) e si deve accedere a prodotto (1 accesso) per scoprire chi è il venditore.

- $NA_r = 1 + 1 = 2$
- $NA_w = 0$
- $C(S) = 1000 \times 1 \times (3 \times 0 + 2) = 2.000$

Schema con ridondanza (S_{rid}):

L'ID del negoziante è già presente nella riga dell'ordine.

- $NA_r = 1$
- $NA_w = 0$
- $C(S_{rid}) = 1000 \times 1 \times (3 \times 0 + 1) = 1.000$

Risultato: La ridondanza dimezza il costo operazionale di lettura ($\Delta C = 2$), garantendo inoltre che il dato rimanga accessibile anche se il prodotto venisse eliminato dal catalogo (dato che la foreign key su prodotto è ON DELETE SET NULL).

III. Lo schema logico e i trigger

Il database è composto da 13 tabelle non tutte però rispettano la terza forma normale, in esse sono stati implementati diversi vincoli di integrità e trigger.

- **cliente**(id, username, password_hash, email, domanda_segreta, risposta_segreta_hash, data_iscrizione)
- **negoziante**(id, username, password_hash, email, domanda_segreta, risposta_segreta_hash, data_iscrizione)
- **categoria**(id, nome, id_categoria_padre*)
- **prodotto**(id, id_negoziante*, id_categoria*, nome, prezzo, descrizione, quantita_disponibile)
- **foto_prodotto**(id, id_prodotto*, percorso_file)
- **carrello**(id, id_cliente*, totale_elementi, totale_prezzo)
- **elemento_carrello**(id, id_carrello*, id_prodotto*, quantita_aggiunta)
- **metodo_pagamento**(id, id_cliente*, tipo, descrizione, dati_pagamento)
- **indirizzo_spedizione**(id, id_cliente*, nome, cognome, telefono, indirizzo, citta, cap, provincia, descrizione)
- **ordine**(id, id_cliente*, id_metodo_pagamento*, id_indirizzo_spedizione*, totale, data_ordine, stato, nome_destinatario, cognome_destinatario, telefono_destinatario, indirizzo_destinatario, citta_destinatario, cap_destinatario, provincia destinatario)
- **elemento_ordine**(id, id_ordine*, id_prodotto*, id_negoziante*, nome_prodotto, prezzo_unitario, quantita_aggiunta)
- **recensione**(id, id_cliente*, id_prodotto*, valutazione, testo, data_recensione)
- **foto_recensione**(id, id_recensione*, percorso_file)

La tabella metodo_pagamento non rispetta prima forma normale, poiché la colonna dati_pagamento contiene valori non atomici: una stringa JSON che racchiude più attributi (numero carta, CVV, intestatario, ecc.). La prima forma normale richiede che ogni campo contenga un valore indivisibile, mentre qui più informazioni sono aggregate in un unico attributo, rendendo necessaria un'operazione di parsing per il loro utilizzo.

La tabella elemento_ordine rispetta la seconda forma normale ma non la terza a causa di una dipendenza transitiva. L'attributo *id_negoziante* dipende infatti da *id_prodotto* (poiché ogni prodotto appartiene a un negoziante) e non direttamente dalla riga dell'ordine. La catena è: id (PK) → id_prodotto → id_negoziante. Ciò introduce una ridondanza voluta per mantenere la storicizzazione del venditore anche in caso di eliminazione del prodotto.

Le tabelle ordine e indirizzo_spedizione rispettano la seconda forma normale ma non la terza a causa di dipendenze transitive sugli attributi geografici. Campi come città, CAP e provincia presentano dipendenze funzionali tra loro (es. id (PK) → CAP → città, provincia), poiché città e provincia derivano dal CAP e non esclusivamente dalla chiave primaria.

Sono stati implementati 23 trigger. Tra i più importanti:

- **tr_crea_carrello_cliente**: crea automaticamente un carrello vuoto per ogni nuovo cliente.
- **tr_aggiorna_carrello_after_delete/insert/update**: aggiornano i totali del carrello (totale_elementi, totale_prezzo) dopo ogni modifica ai prodotti.
- **tr_aggiorna_stock_ordine**: decrementa la quantità disponibile di un prodotto quando un ordine viene confermato.
- **tr_ripristina_stock_annullamento**: ripristina la quantità di un prodotto in magazzino se un ordine viene annullato.
- **tr_recensioni_solo_prodotti_acquistati**: impedisce a un cliente di recensire un prodotto che non ha acquistato e ricevuto.
- **tr_controlla_username/email_cliente/negoziante_after_insert/update**: una serie di trigger che impediscono la creazione di utenti con username o e-mail duplicati tra le tabelle cliente e negoziante.

Nonostante alcune operazioni vengano gestite tramite PHP, ho scelto di mantenere alcuni trigger affinché le modifiche a basso livello siano sempre garantite indipendentemente dall'architettura back-end utilizzata.

Parte II: Il sito web

Scelte di progettazione e programmazione

Il codice è stato scritto con l'obiettivo di essere, mantenibile e sicuro, separando la logica PHP, la struttura HTML, lo stile CSS (con qualche eccezione) e l'interattività JavaScript.

Lato client

- **HTML 5**: L'HTML è stato strutturato usando tag semantici dove possibile (<nav>, <main>, <footer>, etc.).
- **CSS**: L'aspetto grafico è gestito da un unico foglio di stile, con qualche utilizzo di stile inline all'interno dell'HTML. L'estetica è piuttosto minimale e funzionale agli obiettivi del progetto, anche perché non ho particolare esperienza nel web design, specialmente col design responsive.
- **JavaScript e AJAX**: JavaScript è stato utilizzato per migliorare alcuni elementi interattivi all'interno del sito per migliorare alcuni aspetti dell'esperienza utente senza richiedere il ricaricamento completo della pagina. Le chiamate AJAX, implementate con jQuery per semplicità e compattezza, sono state usate per:
    - **Gestione del carrello**: Aggiunta, modifica e rimozione dei prodotti dalla pagina "dettaglio_prodotti.php" e da "carrello.php".
    - **Gestione delle recensioni**: Aggiunta, modifica e rimozione delle recensioni e delle relative foto dalla pagina "dettaglio_prodotti.php".
    - **Controllo disponibilità**: Controllo in tempo reale della disponibilità dell'username e dell'e-mail durante la registrazione.
    - **Recupero della domanda di sicurezza**: Reperimento in tempo reale della domanda di sicurezza scelta dall'utente in fase di registrazione. Tale domanda serve per la procedura del recupero della password

- **Ricerca dei prodotti**: Anziché ricaricare la pagina per effettuare una ricerca, si effettua una chiamata AJAX che restituisce in tempo reale i risultati della ricerca.

Lato server (PHP e MariaDB)

- **PHP e PDO**: Il back-end è stato sviluppato in PHP. L'interazione con il database è gestita tramite l'estensione PDO (PHP Data Objects), scelta rispetto a mysqli per il suo supporto a molteplici DBMS, utile nel caso si decidesse di cambiarne uno in futuro. Pur essendo consapevole che mysqli sia un'alternativa valida, ho preferito adottare subito PDO, anche in previsione di approfondire e utilizzare altri DBMS.
- **Sicurezza**: Quasi ogni query del database viene eseguita tramite prepared statements con parametri nominativi o posizionali. Questo approccio elimina il rischio di attacchi di tipo SQL Injection. Fa eccezione, ad esempio, la query effettuata nel file "index.php", che recupera gli ultimi 12 prodotti caricati nel sito; la scelta è stata fatta con consapevolezza poiché è una query che non prende in input nessun dato dall'utente. Alcune operazioni, come, per esempio, la registrazione sul sito, il caricamento (o la modifica) di un prodotto o di una recensione e l'emissione di un ordine, avvengono tramite transazioni per rispettare i principi ACID (Atomicity, Consistency, Isolation, and Durability).
- **Gestione degli errori**: La connessione tramite PDO è configurata per lanciare eccezioni (PDO::ERRMODE_EXCEPTION), permettendo una gestione degli errori strutturata tramite l'utilizzo dei blocchi try-catch.
- **Organizzazione del codice**: Il codice è stato organizzato per lo più in file modulari (es. config.php per le impostazioni, header.php e footer.php per il layout) favorendo la riusabilità e la manutenibilità.

Sicurezza

Qui di seguito sono illustrare le principali accortezze adoperate per la sicurezza:

- **Prevenzione SQL Injection**: Come già menzionato, l'uso quasi esclusivo dei prepared statements impedisce che le query vengano costruite inserendo direttamente input provenienti dai form, riducendo così il rischio di SQL Injection.
- **Prevenzione Cross-Site Scripting (XSS)**: Qualsiasi dato proveniente dall'utente e destinato ad essere visualizzato in una pagina HTML viene sanificato tramite la funzione di PHP "htmlspecialchars" (ridefinita come "h" per brevità e i flag "ENT_QUOTES" e "ENT_HTML5" e la codifica predefinita in UTF-8). Questa funzione converte alcuni caratteri speciali come per esempio "<" e ">" nelle loro rispettive entità HTML, impedendo l'esecuzione di script malevoli.
- **Gestione delle password e metodi di pagamento**: Le password degli utenti non vengono mai memorizzate in chiaro. Durante la registrazione viene salvato un hash generato con l'algoritmo PASSWORD_DEFAULT tramite la funzione password_hash, mentre in fase di login la verifica viene effettuata con password_verify. Tuttavia, per quanto riguarda i metodi di pagamento, non avendo familiarità con le pratiche adottate in scenari reali, ho scelto di salvare alcuni dati in chiaro nella tabella metodo_pagamento in formato JSON. Sono consapevole che non si tratti di una soluzione professionale, ma è stata adottata considerando la natura didattica del progetto.
- **Protezione delle risorse**: Il codice controlla i permessi dell'utente autenticato. Ad esempio, un cliente non può accedere al pannello del negoziante, e le operazioni di modifica o cancellazione di una risorsa (es. un prodotto, un indirizzo) verificano sempre che tale risorsa appartenga all'utente che sta effettuando la richiesta.
- **Upload sicuro dei file**: L'upload di immagini (per prodotti e recensioni) è protetto da validazioni server-side che controllano il tipo di file (tramite mime_content_type) e la dimensione, prevenendo il

caricamento di file pericolosi o troppo grossi (per un massimo di 5MB per file) e i tipi consentiti sono ".jpg", ".png" e ".gif".

- **Validazione continua degli input**: Per garantire una maggiore sicurezza, in quasi tutti i campi in cui l'utente può inserire dati vengono effettuati controlli tramite espressioni regolari, sia lato front-end (HTML) sia lato back-end (PHP). Ho scelto di non implementare ulteriori validazioni tramite JavaScript, sebbene migliorino l'esperienza utente, ritengo sufficienti i controlli già presenti e, soprattutto, considero le validazioni lato HTML e JavaScript facilmente eludibili. Per questo motivo, la validazione lato server in PHP rimane il punto di riferimento principale e più affidabile.

- **Recupero password tramite domanda di sicurezza**: Non disponendo di alcune tecnologie necessarie per l'invio di e-mail, per aumentare la sicurezza e fornire agli utenti un metodo autonomo per recuperare l'accesso al proprio account, è stato implementato un sistema di recupero password basato su domanda e risposta segreta. La gestione della risposta segreta segue gli stessi standard di sicurezza della password principale. Il processo è integrato in tre aree del sito:

    1. **Registrazione**: Durante la creazione di un nuovo account, all'utente è richiesto obbligatoriamente di scegliere una domanda da un elenco predefinito (es. "In quale città sei nato?") e di fornire una risposta personale (in pratica, è come se fosse una seconda password).

    2. **Processo di recupero**: Se un utente dimentica la propria password, può avviare la procedura di recupero dalla pagina di login. Il recupero si articola in due passaggi:

        - **Identificazione**. L'utente inserisce il proprio username o la propria e-mail. Il sistema recupera e mostra la domanda di sicurezza associata a quell'account.

        - **Verifica e reset**. L'utente deve inserire la risposta corretta alla sua domanda di sicurezza. Se la risposta corrisponde a quella salvata, gli verrà consentito di inserire e confermare una nuova password.

    3. **Gestione profilo**: Un utente già registrato al sito può modificare in qualsiasi momento la propria domanda e risposta di sicurezza dalla pagina del profilo.

Dati di prova

Per la prova del sito, sono stati creati 2 account di tipo cliente e 4 di tipo negoziante.

N.B.: è possibile accedere sia tramite l'username sia tramite e l'e-mail fornita in fase di registrazione, per brevità, di seguito, verranno forniti soltanto gli username, le password e le risposte segrete.

Cliente:

- Username: jack_sparow – Password: Ciao123456 – Risposta segreta: Pirati dei caraibi
- Username: deliaaa – Password: Ciao123456 – Risposta segreta: Palermo

Negoziante:

- Username: angolotecnologico – Password: Ciao123456 – Risposta segreta: Fido
- Username: lacasadimarti– Password: Ciao123456 – Risposta segreta: Pizza
- Username: libreria.delcorso – Password: Ciao123456 – Risposta segreta: Maggio
- Username: nonsolo_sport – Password: Ciao123456 – Risposta segreta: Pisa